


Aufgabe 1:

a) GET /index.html HTTP/1.1

Jede HTTP-Anfrage beginnt mit der Requestline

GET - Methode

/index.html - Pfad zur Ressource

HTTP/1.1 - verwendete Protokollversion

Host: www.hs-furtwangen.de

Verpflichtend!

User-Agent: Mozilla/5.0

Accept: text/html

Accept-Language: de-DE

Connection: close

Optional!

User-Agent - welcher Browser/Client wird gefragt?

Accept - welche Formate akzeptiert der Client?

Accept-Language - bevorzugte Sprache

Connection - ob die TCP Verbindung offen bleiben soll

GET /index.html HTTP/1.1

Host: www.hs-furtwangen.de <- Verpflichtend

User-Agent: Mozilla/5.0 <- Optional

Accept: text/html <- Optional

Accept-Language: de-DE <- Optional

Connection: close <- Optional

b) Ressource existiert nicht:

Statuszeile: HTTP/1.1 404 Not Found

- es wird keine Datei mit dem Namen erkannt!

HTTP/1.1 404 Not Found

Date: Fri, 17 Apr 2026 10:00:00 GMT

Server: Apache/2.4.41

Content-Type: text/html

Content-Lenght: 50

<html><body><h1>404 - Not Found</h1></body></html>

Ressource wurde verschobe:

Statuszeile: HTTP/1.1 301 Moved Permanently

Ressource ist dauerhaft woanderst!

- Der Client soll ab jetzt diese URL nehmen:

Location: /home.html

HTTP/1.1 301 Moved Permanently

Location: /html.home

Date: Fri, 17 Apr 2026 10:00:00 GMT

Server: Apache/2.4.41

Content-Lenght: 50

Keine Berechtigung:

Statuszeile: HTTP/1.1 403 Forbidden

Der Server hat die Datei aber du darfst sie nicht sehen!

HTTP/1.1 403 Forbidden

Date: Fri, 17 Apr 2026 10:00:00 GMT

Server: Apache/2.4.41

Content-Type: text/html

Content-Length: 60

<html><body><h1>403 - Forbidden</h1></body></html>

c)

1) Statuszeile: **HTTP/1.1 200 OK**

- HTTP/1.1 - verwendete Protokollversion
- 200 OK - Anfrage ist erfolgreich, Ressource wird übertragen

2) Date: **Mon, 12 Oct 2025 10:00:00 GMT**

- Zeitpunkt für wann der Server die Antwort erzeugt hat

3) Server: **Server: Apache/2.4.41 (Ubuntu)**

- Server Software + Version

4) Content-Type: **Content-Type: text/html; charset=UTF-8**

- MIME-Typ der Übertragungsdaten
- Browser weiß dadurch wie er den Inhalt darstellen soll
- charset=UTF-8 - Zeichensatz

5) Content-Length: **Content-Length: 138**

- Größe des HTML Dokuments in Bytes
- Browser weiß wann die Übertragung vollständig ist

6) Set-Cookie: **Set-Cookie: sessionId=abc123**

- Server weist dem Client eine Session ID zu
- Browser speichert Cookie
- Wird bei jeder Anfrage automatisch mitgeschickt

7) Body

Der HTML Body der Webseite

d)

POST /formular HTTP/1.1

Post wird verwendet wenn:

- Daten an den Server gesendet werden
- ein Formular abgeschickt wird
- etwas verändert wird z.B. Registrierung, Login oder Bestellung

Host: www.hs-furtwangen.de

Content-Type: application/x-www-form-urlencoded

Content-Length: 35

name=Robin&email=robindambach09%40gmail.com

POST /formular HTTP/1.1

Host: www.hs-furtwangen.de

Content-Type: application/x-www-form-urlencoded

Content-Length: 35

name=Robin&email=robindambach09%40gmail.com

Aufgabe 2:

a)

Ein Web Proxy ist ein Vermittler zwischen Client (z.b. Browser) und dem Internet.

Was macht ein Proxy?

- Anfragen weiterleiten: Client - Proxy - Webserver
- Inhalte zwischenspeichern: Caching (z.b. bei viel Traffic)
- Filtern/Sperren: Jugendschutz, Filterrichtlinien, Blockieren
- Anonymisieren: Webserver sieht nur die IP des Proxys nicht Client
- Sicherheitskontrolle: Malware Filter, Logging, Schutz vor Angriffen

Wo wird ein Proxy eingesetzt?

- Unternehmen
- Schulen/Hochschulen/Uni
- Öffentliche WLAN`s
- Länder mit Internet Zensur

Vorteile eines Proxys?

- Weniger bandbreite durch Caching
Der Proxy speichert häufig genutzte Inhalte - schnellere Ladezeit
- Mehr Sicherheit
- Anonymisierung
- Protokollierung

b)

Wie prüft der Cache die Gültigkeit einer Ressource?

Ein Cache speichert Webseiten, Bilder, CSS, JS...

Damit er weiß ob seine Kopie noch aktuell ist fragt er beim server nach.

Es gibt da 2 Methoden

Mechanismus A: (If Modified Since)

Der Cache fragt:

"Wurde die Datei seit diesem Datum geändert?"

Mechanismus B: (If None Match(ETag))

Der Cache fragt:

"Hat sich der Inhalt geändert?(Vergleich über eindeutige ID)"

Mechanismus A reicht eigentlich aus.

HTTP Anfrage des Caches:

Mechanism A:

GET /bild.png HTTP/1.1

Host: www.hs-furtwangen.de

If-Modified-Since: Fri, 17 Apr 2026 12:00:00 GMT

Antwort wenn die Ressource unverändert ist:

HTTP/1.1 304 Not Modified

Date: Fri, 17 Apr 2026 12:00:00 GMT

Antwort wenn die Ressource Aktualisiert wurde:

HTTP/1.1 200 OK

Date: Fri, 17 Apr 2026 12:00:00 GMT

Last-Modified: Fri, 17 Apr 2026 11:00:00 GMT

Content-Type: image/png

Content.Length: 12345

(binary data)

c)

1. Ausgangsanfrage:

GET /webshop/cart HTTP/1.1

Host: www.furtwangen-onlineshop.de

Der Client ruft den Warenkorb auf

Der Server merkt:

- Der Benutzer hat noch keine Session
- er braucht eine Session ID um den Warenkorb zu speichern.

2. Server setzt ein Cookie

HTTP/1.1 200 OK

Set-Cookie: sessionId=xyz987; Path=/; HttpOnly

Content-Type: text/html

Content-Length: 120

- Set-Cookie: Browser soll Cookies speichern
- sessionId=xyz987: eindeutige ID für den Besucher
- Path=/: Cookie gilt für die ganze Webseite
- HttpOnly: Cookie kann nicht von JS gelesen werden (Sicherheit)

3. Browser sendet Cookie bei jeder Weiteren Anfrage

GET /webshop/cart HTTP/1.1

Host: www.furtwangen-onlineshop.de

Cookie: sessionId=xyz987

- Der Server erkennt das es der gleiche Benutzer ist!

4. Beispiel: Benutzer legt ein Produkt in den Warenkorb

Anfrage:

POST /webshop/add HTTP/1.1

Host: www.furtwangen-onlineshop.de

Cookie: sessionId=xyz987

Content-Type: application/x-www-form-urlencoded

Content-Length: 20

itemId=42&qty=1

Antwort:

HTTP/1.1 200 OK

Content-Type: application/json

```
{"message": "Item added to cart"}
```

Der Server Speichert den Warenkorb unter der Session ID

5. Warum helfen Cookies bei Sitzungen?

Weil HTTP zustandslos ist.

Das bedeutet:

- Jede Anfrage ist unabhängig
- Der Server weiß nicht automatisch wer du bist
- Ohne Cookies würde der Warenkorb bei jeder Anfrage verloren gehen

Cookies lösen das Problem:

- Der Server gibt dem Benutzer eine Session ID
- Der Browser sendet sie bei jeder Anfrage zurück
- Der Server kann deinen Warenkorb, Login, Einstellungen... zuordnen

Datei Finden:

cd "C:\Users\robin\Documents\http-test"

Starten:

python -m http.server 8000 --protocol HTTP/1.0

Port und Aufrufen:

http://localhost:8000

Telnet:

telnet localhost 8000

GET /index.html HTTP/1.0

Host: local host

Dann kommt diese ausgabe bei raus:

```
Administrator: Windows PowerShell

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.11.13
Date: Sun, 03 May 2026 09:11:00 GMT
Content-type: text/html
Content-length: 1368
Last-Modified: Sun, 03 May 2026 08:23:21 GMT

<!doctype html>
  <html lang="de">
    <head>
      <meta charset="UTF-8" />
      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
      <title>Aufgabe 02: HTTP-Versionen.</title>
      <style>
        body {
          font-family: sans-serif;
          margin: 2rem;
          line-height: 1.6;
        }
        .placeholders {
          display: flex;
          gap: 1rem;
          flex-wrap: wrap;
          margin-top: 1.5rem;
        }
        .placeholders img {
          width: 280px;
          height: 180px;
          object-fit: cover;
          border: 1px solid #bbb;
          border-radius: 6px;
        }
      </style>
    </head>
    <body>
      <h1>Hello World</h1>
      <h2>Internetprotokolle: Aufgabe 02 - HTTP</h2>
      <p>
        Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
        tempor incididunt ut labore et dolore magna aliqua.
      </p>
      <div class="placeholders">
        
        
      </div>
      <p>
        Images from <a href="https://www.pexels.com/">Pexels</a> with Creative Commons Zero (CC0) license.
      </p>
    </body>
  </html>
```

PS C:\windows\system32>

Datei anlegen im Ordner http-test

-> request.txt datei

Inhalt:

GET /index.html HTTP/1.0

Host: localhost

Dann

Windows + R

cmd eingeben

Pfad auf Ordner Stellen(http-test)

cd C:\Users\robin\Documents\http-test

Dann Ncat anfrage:

ncat localhost 8000 < request.txt

Ergebniss:

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\robin>cd C:\Users\robin\Documents\http-test

C:\Users\robin\Documents\http-test>ncat localhost 8000 < request.txt
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.13.13
Date: Sun, 03 May 2026 09:41:42 GMT
Content-type: text/html
Content-Length: 1368
Last-Modified: Sun, 03 May 2026 08:23:21 GMT

<!doctype html>
<html lang="de">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Aufgabe 02: HTTP-Versionen.</title>
    <style>
      body {
        font-family: sans-serif;
        margin: 2rem;
        line-height: 1.6;
      }

      .placeholders {
        display: flex;
        gap: 1rem;
        flex-wrap: wrap;
        margin-top: 1.5rem;
      }

      .placeholders img {
        width: 280px;
        height: 180px;
        object-fit: cover;
        border: 1px solid #bbb;
        border-radius: 6px;
      }
    </style>
  </head>
  <body>
    <h1>Hello World</h1>
    <h2>Internetprotokolle: Aufgabe 02 - HTTP</h2>
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua.
    </p>

    <div class="placeholders">
```

CURL:

Curl mit Header:

```
curl -i http://localhost:8000/index.html
```

Ergebniss:

```
C:\windows\system32\cmd.exe X + v
C:\Users\robin>curl -i http://localhost:8000/index.html
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.13.13
Date: Sun, 03 May 2026 09:46:39 GMT
Content-type: text/html
Content-Length: 1368
Last-Modified: Sun, 03 May 2026 08:23:21 GMT

<!doctype html>
<html lang="de">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Aufgabe 02: HTTP-Versionen.</title>
    <style>
      body {
        font-family: sans-serif;
        margin: 2rem;
        line-height: 1.6;
      }

      .placeholders {
        display: flex;
        gap: 1rem;
        flex-wrap: wrap;
        margin-top: 1.5rem;
      }

      .placeholders img {
        width: 280px;
        height: 180px;
        object-fit: cover;
        border: 1px solid #bbb;
        border-radius: 6px;
      }
    </style>
  </head>
  <body>
    <h1>Hello World</h1>
    <h2>Internetprotokolle: Aufgabe 02 - HTTP</h2>
    <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
      tempor incididunt ut labore et dolore magna aliqua.
    </p>

    <div class="placeholders">
      
      
    </div>
    <p>
      Images from <a href="https://www.pexels.com/">Pexels</a> with Creative Commons Zero (CC0) license.
    </p>
  </body>
</html>
```

Aufgabe 4:

Der Browser verwendet HTTP/1.1 für die Kommunikation mit dem Server!

Wie viele Antworten und Anfragen werden gesendet?

3 Anfragen
3 Antworten

TCP Verbindungen:

3 Stück

SYN

SYN ACK

ACK

HTTP/1.0

- Keine persistenten Verbindungen
- Für jede Datei neue TCP-Verbindung
- Mehr Overhead

HTTP/1.1

- Keep-Alive standardmäßig aktiv
- Mehrere Dateien über eine einzige Verbindung
- Effizienter und schneller

Aufgabe 5:

Liste aller Studenten:

curl -X GET http://localhost:8001/students

```
PS C:\windows\system32> curl.exe -X GET http://localhost:8001/students
>>
[{"matriculation_number":"123456","name":"Max Mustermann","study_course":"Informatik"},
{"matriculation_number":"654321","name":"Erika Musterfrau","study_course":"Mathematik"}]
PS C:\windows\system32> _
```

Tom Tester anlegen:

```
curl -X POST http://localhost:8001/students -H "Content-Type: application/json" -d "{\"name\":\"Tom Tester\", \"matriculation_number\":\"987654\", \"study_course\":\"Erdkunde\"}"
```

(Powershell hat probleme gemacht auf cmd gewechselt!)

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\robin> cd C:\Users\robin\Documents\dockerfile

C:\Users\robin\Documents\dockerfile> curl -X POST http://localhost:8001/students -H "Content-Type: application/json"
-d "{\"name\":\"Tom Tester\", \"matriculation_number\":\"987654\", \"study_course\":\"Erdkunde\"}"
{"message":"Student added successfully"}

C:\Users\robin\Documents\dockerfile>
```

```
Schöndruck ✓
[
  {
    "matriculation_number": "123456",
    "name": "Max Mustermann",
    "study_course": "Informatik"
  },
  {
    "matriculation_number": "654321",
    "name": "Erika Musterfrau",
    "study_course": "Mathematik"
  },
  {
    "matriculation_number": "987654",
    "name": "Tom Tester",
    "study_course": "Erdkunde"
  }
]
```

Studiengang von Tom ändern!

```
curl -X PUT http://localhost:8001/students/987654 -H "Content-Type: application/json" -d '{"name": "Tom Tester", "study_course": "Informatik"}'
```

```
C:\Users\robin\Documents\dockerfile> curl -X PUT http://localhost:8001/students/987654 -H "Content-Type: application/json" -d '{"name": "Tom Tester", "study_course": "Informatik"}'
{"message": "Student updated successfully"}
C:\Users\robin\Documents\dockerfile>
```

```
"GET /students HTTP/1.1" 200 -
"GET /favicon.ico HTTP/1.1" 404 -
"GET /students HTTP/1.1" 200 -
"POST /students HTTP/1.1" 415 -
"GET /students HTTP/1.1" 200 -
"POST /students HTTP/1.1" 400 -
"POST /students HTTP/1.1" 400 -
"POST /students HTTP/1.1" 201 -
"GET /students HTTP/1.1" 200 -
"PUT /students/987654 HTTP/1.1" 200 -
"GET /students HTTP/1.1" 200 -
"DELETE /students/987654 HTTP/1.1" 200 -
"GET /students HTTP/1.1" 200 -
"GET /students HTTP/1.1" 200 -
```

```
localhost:8001/students
Schöndruck ✓
[
  {
    "matriculation_number": "123456",
    "name": "Max Mustermann",
    "study_course": "Informatik"
  },
  {
    "matriculation_number": "654321",
    "name": "Erika Musterfrau",
    "study_course": "Mathematik"
  },
  {
    "matriculation_number": "987654",
    "name": "Tom Tester",
    "study_course": "Informatik"
  }
]
```

Tom Löschen:

```
curl -X DELETE http://localhost:8001/students/987654
```

```
C:\Users\robin\Documents\dockerfile> curl -X DELETE http://localhost:8001/students/987654
{"message": "Student deleted successfully"}
C:\Users\robin\Documents\dockerfile>
```

```
Schöndruck ✓
[
  {
    "matriculation_number": "123456",
    "name": "Max Mustermann",
    "study_course": "Informatik"
  },
  {
    "matriculation_number": "654321",
    "name": "Erika Musterfrau",
    "study_course": "Mathematik"
  }
]
```

Get erneut ausführen (Kontrolle):

```
curl -X GET http://localhost:8001/students
```

```
C:\Users\robin\Documents\dockerfile> curl -X GET http://localhost:8001/students
[{"matriculation_number": "123456", "name": "Max Mustermann", "study_course": "Informatik"}, {"matriculation_number": "654321", "name": "Erika Musterfrau", "study_course": "Mathematik"}]
```

zum wieder starten:

```
docker run --rm -p 8001:8001 student_server
```